

《计算概论》 程序设计部分

字符数组与字符串

董 豪

北京大学 计算机学院 前沿计算研究中心

hao.dong@pku.edu.cn

字符数组的定义

```
#include<iostream>
using namespace std;
int main()
{
        char a[10] = { 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j' };
        for (int i = 0; i < 10; i++)
                cout << a[i];
        return 0;
}
```

a	b	c	d	e	f	g	h	i	j
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

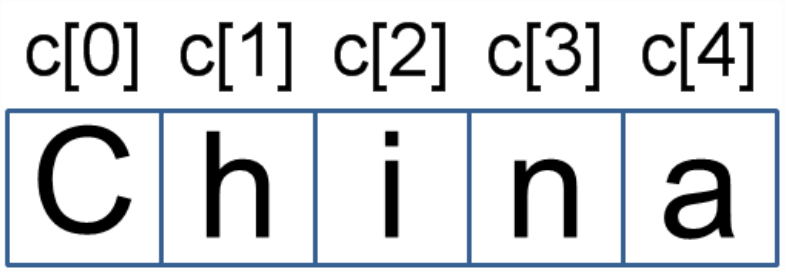
字符数组的定义

```
#include<iostream>
using namespace std;
int main()
{
        char a[10] = { 'a', 'b', 'c', 'd', 'e'};
        for (int i = 0; i < 10; i++)
                cout << a[i];
        return 0;
}
```

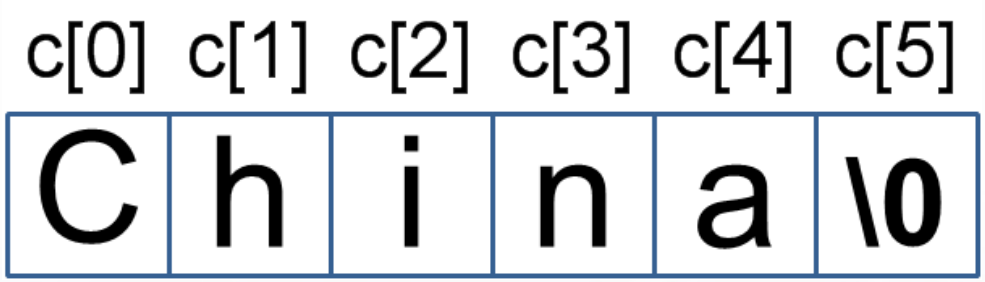
a	b	c	d	e	\0	\0	\0	\0	\0
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

字符数组的初始化

字符数组 **char c[ ] = {‘C', ‘h', ‘i', ‘n', ‘a'};**



字符串 **char c[ ] = “China”;**



认识一下字符串

“China”;



char c[5] = “China”; X

char c[6] = “China”; O

char c[] = “China”; O

关于赋值

- **只可以：**
  - 在数组定义并初始化的时候：

    char c[6] = “China”;

    char c[ ] = “China”;
- **不可以：**
  - 不能用赋值语句将一个字符串常量或字符数组直接赋给另一个字符数组。

str1[ ]= “China”; **(赋值，不合法)**

str1 = “China”; **(赋值，不合法)**

str2 = str1; **(赋值，不合法)**

正确的赋值方式

```
#include<iostream>
using namespace std;
int main()
{
        char str1[] = "C++ language", str2[20];
        int i = 0;
        while (str1[i] != '\0')
        {
                str2[i] = str1[i];
                i++;
        }
        str2[i] = '\0';
        cout << "String1:" << str1 << endl;
        cout << "String2:" << str2 << endl;
        return 0;
}
```

```
String1:C++ language
String2:C++ language
```

正确的赋值方式

```
#include<iostream>
using namespace std;
int main()
{
        char str1[] = "C++ language", str2[20];
        int i = 0;
        do{
                str2[i] = str1[i];
                i++;
        }while (这里写什么？ )
        cout << "String1:" << str1 << endl;
        cout << "String2:" << str2 << endl;
        return 0;
}
```

```
String1:C++ language
String2:C++ language
```

正确的赋值方式

```
#include<iostream>
using namespace std;
int main()
{
        char str1[] = "C++ language", str2[20];
        int i = 0;
        do{
                str2[i] = str1[i];
                i++;
        }while (str1[i-1] != '\0')
        cout << "String1:" << str1 << endl;
        cout << "String2:" << str2 << endl;
        return 0;
}
```

```
String1:C++ language
String2:C++ language
```

字符串数组

- 利用二维数组存储多个字符串

char weekday[7][11] = {“Sunday", "Monday", "Tuesday", "Wednesday", “Thursday", “Friday", “Saturday"};

S	u	n	d	a	y	\0	\0	\0	\0	\0
M	o	n	d	a	y	\0	\0	\0	\0	\0
T	u	e	s	d	a	y	\0	\0	\0	\0
W	e	d	n	e	s	d	a	y	\0	\0
T	h	u	r	s	d	a	y	\0	\0	\0
F	r	i	d	a	y	\0	\0	\0	\0	\0
S	a	t	u	r	d	a	y	\0	\0	\0



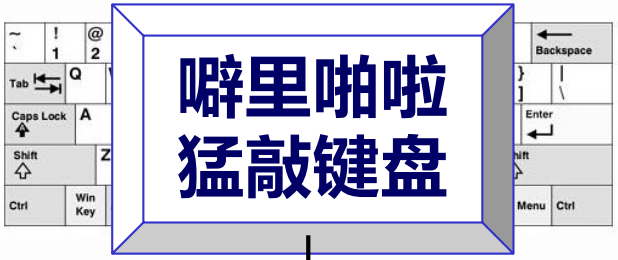
字符/字符数组/字符串的输入与输出

I. 引子：输入的过程

II. 一个字符的输入与输出

III. 字符数组/字符串的输入与输出

认识一下：输入缓冲区



Sa 撒[illegible]西了，
输[illegible]使用>>
操作符时输[illegible]
Tab、回车就停止了。



缓冲区

※ cin使用>>时会忽略空格、Tab和回车。

str: "How?"

挂起中

Suspended

```
while (true) {
    cin >> str;
    cout << [illegible]
}
```





用cin输入数据

```
int a, b ;
cin>>a>>b; // 从键盘输入 21  22↙
```

```
int a, b ;
cin>>a>>b; // 从键盘输入 21  abc ↙
```

```
int a, b, c ;
cin>>a>>b>>c;   // 从键盘输入 21  22 ↙
                // 23 ↙
```



C语言输入输出

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");  // 输出字符串
    printf("整数: %d\n", 42);    // 输出整数
    printf("浮点数: %.2f\n", 3.14159);  // 输出保留两位小数的浮点数
    printf("字符: %c\n", 'A');  // 输出字符
    printf("十六进制: %x\n", 255);  // 十六进制输出
    printf("\n");
    float fnum;
    scanf("%f", &fnum);  // 输入浮点数
    printf("你输入的浮点数是: %.2f\n", fnum);
    return 0;
}
```

用cin输入数据

```
#include<iostream>
using namespace std;
int main() {
        float grade;
        cout << "enter grade : ";
        while (cin >> grade)      //能从cin流读取数据
        {
                if (grade >= 85)
                        cout << grade << "GOOD!" << endl;
                if (grade < 60)
                        cout << grade << "fail!" << endl;
                cout << "enter grade:";
        }
        return 0;
}
```



流：C++统一管理IO的方法

- 什么是流
  - 由于程序开发的需要，需要在程序间传递数据；
  - 这些数据按顺序组成由若干字节组成的字节序列；
  - 传递时，在内存中开辟一个内存缓冲区，用来存放流动中的数据。
  - 缓冲区中的数据就是流；
- 流的分类：
  - 输出流：从内存中送一串字节到显示器、打印机；
  - 输入流：从键盘送一串字节到内存；
  - 文件流：从内存送一串字节到外存；
- 与流相关的运算符：输出<<, 输入>>

流：C++统一管理IO的方法

- 统一管理不同数据类型输入输出
  - 定义std::ostream& **operator<<**(std::ostream&, T rhs)
  - 定义std::ostream& **operator>>**(std::ostream&, T& rhs)





常用输入输出流

- iostream中的输入输出流
  - 标准输入流cin:
    - 从标准输入设备输入到程序的数据流;
  - 标准输出流cout:
    - 从程序输出到标准输出设备的数据流;
  - 出错信息流cerr和clog流:
    - 向显示器输出出错信息;
- fstream中定义的文件输入输出流



cin流对象

- cin流对象
  - 从标准输入设备(键盘)获取数据；
  - 程序中的变量通过流提取运算符“>>”从流中提取数据；
  - 流提取符“>>”从流中提取数据时通常跳过输入流中的空格、tab键、换行符等空白字符；
  - 遇到无效字符或文件结束标志，停止提取；
  - 只有在输入完数据再按回车键后，该行数据才被送入键盘缓冲区，形成输入流，提取运算符“>>”才能从中提取数据；

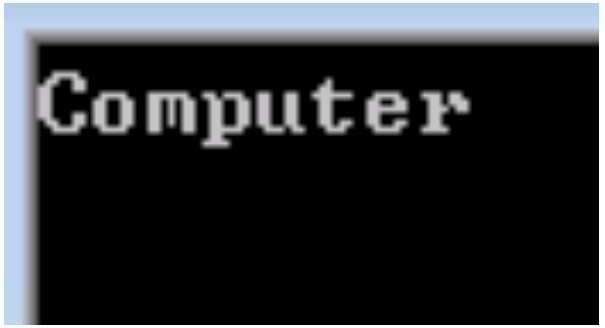


cout流对象

- cout (console output，控制台输出)
  - cout流在内存中对应开辟了一个缓冲区，用来存放流中的数据，
  - 当向cout流插入一个endl时，不论缓冲区是否已满，都立即输出流中所有数据，并加入一个换行符，然后刷新流(清空缓冲区)。
  - 用“cout<<”输出基本类型的数据时，不必考虑数据是什么类型，系统会判断数据的类型;

用 cout 输出字符数组

```
#include <iostream>
using namespace std;
int main()
{
        char a[10] = "Computer";
        cout << a;
        return 0;
}
```



C	o	m	p	u	t	e	r	\0	\0
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

用 cout 输出字符数组

```
#include <iostream>
using namespace std;
int main()
{
        char weekday[7][11] = { "Sunday", "Monday",
"Tuesday", "Wednesday", "Thursday", "Friday",
"Saturday" };
        for (int i = 0; i < 7; i++)
                cout << weekday[i] << endl;
        return 0;
}
```

S	u	n	d	a	y	\0	\0	\0	\0	\0
M	o	n	d	a	y	\0	\0	\0	\0	\0
T	u	e	s	d	a	y	\0	\0	\0	\0
W	e	d	n	e	s	d	a	y	\0	\0
T	h	u	r	s	d	a	y	\0	\0	\0
F	r	i	d	a	y	\0	\0	\0	\0	\0
S	a	t	u	r	d	a	y	\0	\0	\0

```
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
```

用 cout 输出字符数组

```
#include <iostream>
using namespace std;
int main()
{
        char weekday[7][11] = { "Sunday", "Monday",
"Tuesday", "Wednesday", "Thursday", "Friday",
"Saturday" };
        cout << weekday<< endl;
        return 0;
}
```

S	u	n	d	a	y	\0	\0	\0	\0	\0
M	o	n	d	a	y	\0	\0	\0	\0	\0
T	u	e	s	d	a	y	\0	\0	\0	\0
W	e	d	n	e	s	d	a	y	\0	\0
T	h	u	r	s	d	a	y	\0	\0	\0
F	r	i	d	a	y	\0	\0	\0	\0	\0
S	a	t	u	r	d	a	y	\0	\0	\0

```
0x7ffe20a436c0
```

用 cout 输出字符数组

```
#include <iostream>
using namespace std;
int main()
{
        int a[8] = { 1,2,3,4,5,6 };
        cout << a;
        return 0;
}
```

```
0x7ffc68453f40
```

使用控制符控制输出格式

操 作 符	描 述	备 注
dec	按十进制输出	常量控制符 在 iostream.h 中
hex	按十六进制输出	
oct	按八进制输出	
endl	插入换行符，并刷新流	
ends	插入空字符	
setbase(n)	设置整数基数为 n	函数控制符 在 iomanip.h 中
setfill(c)	设置填充字符 c	
setprecision(n)	设置实数物精度为 n	
setw(n)	设置字段宽度为 n	
setiosflags(ios::fixed)	设置浮点数以固定的小数位数显示	
setiosflags(ios::scientific)	设置浮点数以科学记数法显示	
setiosflags(ios::left)	输出数据左对齐	
setiosflags(ios::right)	输出数据右对齐	
setiosflags(ios::skipws)	忽略前导的空格	
setiosflags(ios::uppercase)	十六进制数大写输出	
setiosflags(ios::lowercase)	十六进制数小写输出	
setiosflags(ios::showpos)	输出正数时给出“+”号	
resetioflags()	终止已设置的输出格式状态	

```
#include<iostream>
#include<iomanip>
using namespace std;
int main() {
    int a, b;
    cout<<"以八进制格式输入整数：\na=0";
    cin >> oct >> a;
    cout << "b=0";
    cin >> b; //仍然保持八进制设置
    cout<<"以十六进制显示整数 a=0x" << hex << a << ", b=0x" << b << endl;
    cout<<"以十进制显示整数   a=" << dec << a << ", b=" << b << endl;
    cout<<"以八进制显示整数   a=0" << oct << a << ", b=0" << b << endl;
    cout<<resetiosflags(ios::oct|ios::showbase);
    cout<<setiosflags(ios::hex|ios::showbase);
    cout<<"以十六进制显示整数 a="<<a<<",b="<<b<<endl;
    cout<<resetiosflags(ios::hex|ios::showbase);
    cout<<setiosflags(ios::dec|ios::showbase);
    cout<<"以十进制显示整数  a="<<a<<",b="<<b<<endl;
    cout<<resetiosflags(ios::dec|ios::showbase);
    cout<<setiosflags(ios::oct|ios::showbase);
    cout<<"以八进制显示整数  a="<<a<<",b="<<b<<endl;
    return 0;
}
```

```
以八进制格式输入整数:
a=0 20
b=0 10
以十六进制显示整数 a=0x10,  b=0x8
以十进制显示整数    a=16,  b=8
以八进制显示整数    a=020,  b=010
以十六进制显示整数 a=0x10, b=0x8
以十进制显示整数   a=16, b=8
以八进制显示整数   a=020, b=010
Press any key to continue
```

```
#include<iostream>
#include<iomanip>
using namespace std;
int main() {
    float x = 35.54767f;
    int i;
    cout << x << endl;
    cout << setprecision(7)<<x<< endl;
    cout << setiosflags(ios::fixed);
    for (i = 0; i < 6; i++)
        cout<<setprecision(i)<<x<<endl;
    cout<<resetiosflags(ios::fixed)<<endl;
    cout<<setiosflags(ios::scientific);
    for (i = 0; i < 6; i++)
        cout<<setprecision(i)<<x<<endl;
    return 0;
}
```

```
35.5477
35.54767
36
35.5
35.55
35.548
35.5477
35.54767

4e+01
3.6e+01
3.55e+01
3.555e+01
3.5548e+01
3.55477e+01
```

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{    int i;
     double values[ ] = {1.23, 35.36, 653.7, 4358.24};
     char *names[ ] = {"Zoot", "Jimmy", "Al", "Stan"};
//   cout << setiosflags(ios::scientific);
//   cout << setiosflags(ios::fixed);
     for(i=0; i<4; i++)
     {
        cout << setiosflags(ios :: left)
             << setw(6) << names[i]
             << resetiosflags(ios :: left)
             << setw(10) << setprecision(1) << values[i] << endl;
     }
     return 0;
}
```

```
Zoot      1.2e+00
Jimmy     3.5e+01
Al        6.5e+02
Stan      4.4e+03
```

```
Zoot          1.2
Jimmy        35.4
Al          653.7
Stan       4358.2
```

控制输出宽度（1）

```
#include <iostream>
using namespace std;
int main()
{
    int i;
    double values[ ] = {1.23, 35.36, 653.7, 4358.24};
    for(i=0; i<4; i++)
    {
        cout.width(10);
        cout << values[i]<<endl;
    }
    return 0;
}
```

```
      1.23
     35.36
     653.7
   4358.24
Press any key to continue
```

控制输出宽度（2）

```
#include <iostream>
using namespace std;
int main()
{
    int i;
    double values[ ] = {1.23, 35.36, 653.7, 4358.24};
    for(i=0; i<4; i++)
    {
        cout.width(10);
        cout.fill('*');
        cout << values[i] << endl;
    }
    return 0;
}
```

```
*******1.23
******35.36
******653.7
***4358.24
Press any key to continue
```

控制输出宽度（3）

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int i;
    double values[ ] = {1.23, 35.36, 653.7, 4358.24345678912345};
    char *names[ ] = {"Zoot", "Jimmy", "Al", "Stan"};
    for(i=0; i<4; i++)
    {
      cout << setw(6) << names[i] << setw(10) <<
                                  setprecision(15)<<values[i] << endl;
    }
    return 0;
}
```

```
  Zoot       1.23
 Jimmy      35.36
    Al      653.7
  Stan4358.24345678912
Press any key to continue
```

控制输出对齐（1）

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    int i;
    double values[ ] = {1.23, 35.36, 653.7, 4358.24};
    char *names[ ] = {"Zoot", "Jimmy", "Al", "Stan"};
    for(i=0; i<4; i++)
    {
        cout << setiosflags(ios::left)
             << setw(6) << names[i]
             << resetiosflags(ios::left)
             << setw(10) << values[i] << endl;
    }
    return 0;
}
```

```
Zoot          1.23
Jimmy        35.36
Al           653.7
Stan      4358.24
Press any key to continue
```

控制输出对齐（2-1）

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    for(int i=0; i<10; i++)
        cout << i;
    cout << endl;
    for(int x=1; x<=10; x++)
        cout << setfill('*') << setw(x) << "*" << endl ;
    return 0;
}
```

```
0123456789
*
**
***
****
*****
******
*******
********
*********
**********
```

控制输出对齐（2-2）

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    for(int i=0; i<10; i++)
        cout << i;
    cout << endl;
    for(int x=1; x<=10; x++)
        cout << setfill('*') << setw(x) << "-" << endl ;
    return 0;
}
```

```
0123456789
-
*-
**-
***-
****-
*****-
******-
*******-
********-
*********-
```

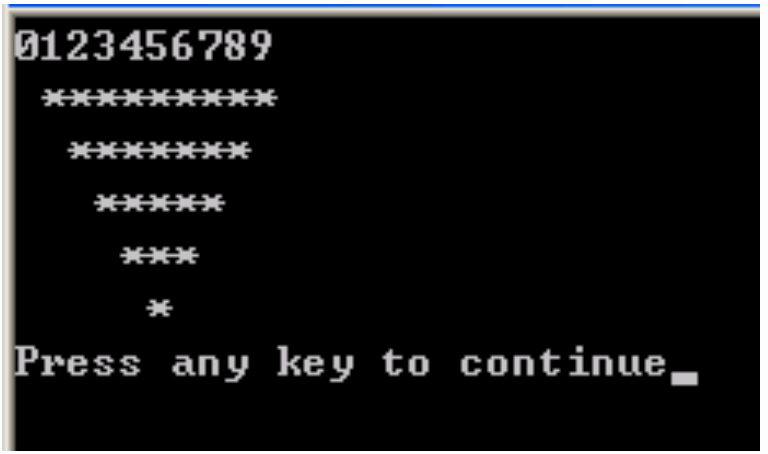
控制输出对齐（3）

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
  for(int i=0; i<10; i++) cout<<i;
  cout << endl;
  for(int x=1; x<=9; x++)
   cout << setfill(‘ ’) << setw(10-x)
        << ‘ ’ << setfill(‘*’) << setw(x) << “*” << endl;
  cout << “**********” <<endl;
  return 0;
}
```



控制输出对齐（4）

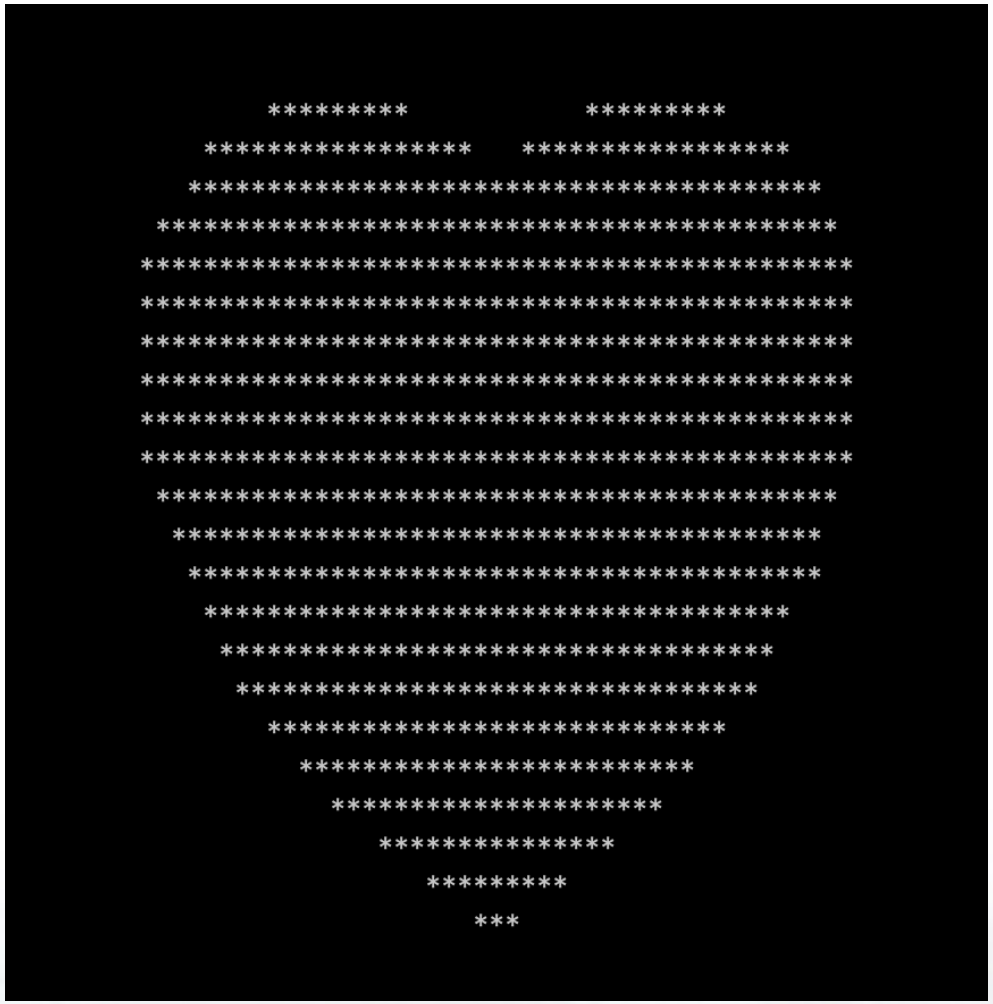
```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    for(int i=0; i<=9; i++) cout<<i;
    cout << endl;
    for(int x=1; x<=5; x++)
        cout << setfill(‘ ’) << setw(x) << “ ” << setfill('*')
                    << setw(11-2*x) << “*” << endl;
    return 0;
}
```





例1 如何输出爱心

- 尝试采用最简短的代码





（1）一个字符的输入

方法一：直接用cin输入字符

```
#include <iostream>
using namespace std;
int main()
{
        char c;
        cout << "enter a sentence:" << endl;
        while(cin >> c)
                cout << c; //每次输出一个字符、跳过空格
        return 0;
}
```

```
enter a sentence:
abc def g
abcdefg
```

方法一：直接用cin输入字符

```
#include <iostream>
using namespace std;
int main()
{
        char c;
        cout << "enter a sentence:" << endl;
        while(cin >> c)
                cout << '-' << c;
        return 0;
}
```

```
enter a sentence:
abc def g
-a-b-c-d-e-f-g
```

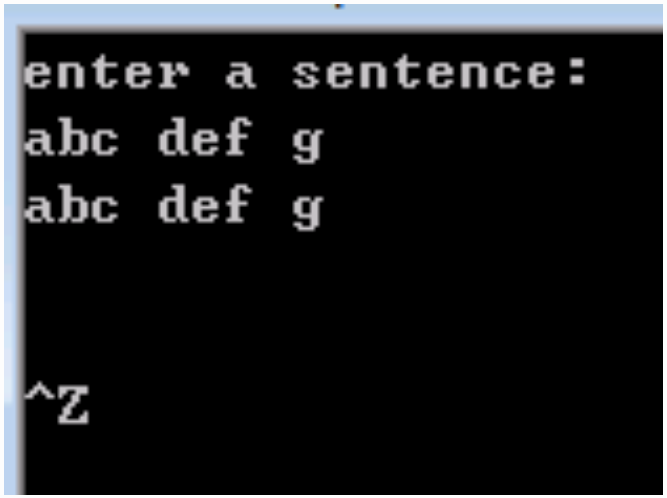


方法二： cin.get

- cin.get( )函数
  - 可以用于读入一个字符；
  - 2种形式: 无参数，一个参数。
    - cin.get( )
    - cin.get(char)

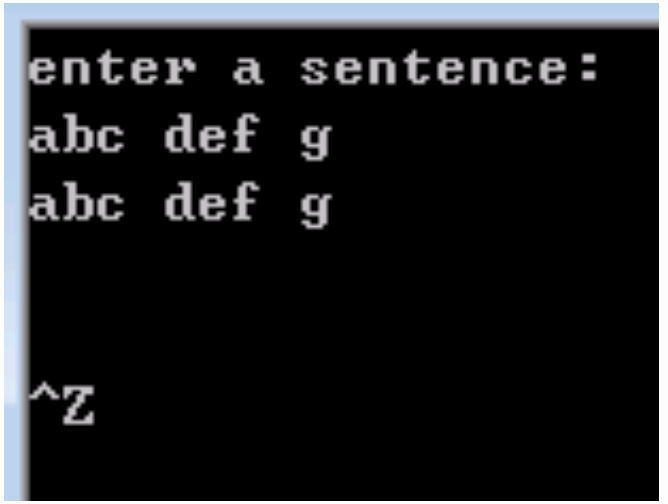
方法二：用 cin.get( ) 输入字符

```
#include <iostream>
using namespace std;
int main()
{
        char c;
        cout << "enter a sentence:" << endl;
        while ((c = cin.get()) != EOF)
                cout<<c; ; //每次一个字符，不跳过空格/回车
        return 0;
}
```



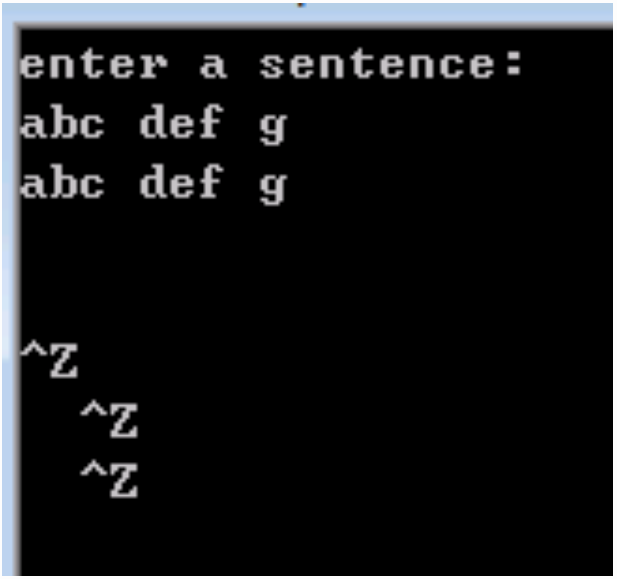
方法三：用 cin.get(char) 输入字符

```
#include <iostream>
using namespace std;
int main()
{
        char c;
        cout << "enter a sentence:" << endl;
        while (cin.get(c))
                cout << c; //每次一个字符，不跳过空格/回车
        return 0;
}
```



方法四：用 getchar( ) 输入字符

```
#include <iostream>
using namespace std;
int main()
{
        char c;
        cout << "enter a sentence:" << endl;
        while (c = getchar()) //不跳过任何字符
                cout<<c;
        return 0;
}
```





小结

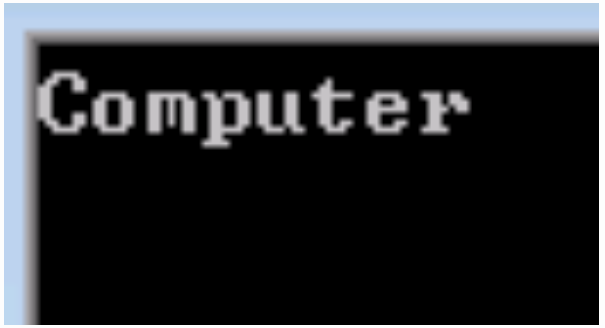
- cin 缓冲区里读取内容时，跳过空格和回车
- cin.get 不跳过空格和回车，只有control+Z才能
- getchar 除了空格和回车，control+Z也不跳过



(2) 一串字符的输出

用 cout 输出字符数组

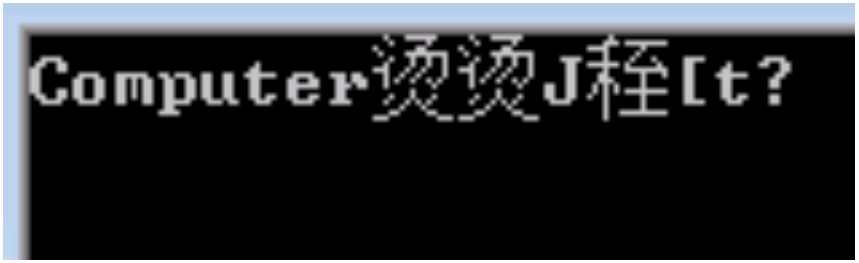
```
#include <iostream>
using namespace std;
int main()
{
    char a[10] = "Computer";
    cout << a;
    return 0;
}
```



C	o	m	p	u	t	e	r	\0	\0
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

用 cout 输出字符数组

```
#include <iostream>
using namespace std;
int main()
{
    char a[8] = { 'C', 'o', 'm', 'p', 'u', 't', 'e', 'r' };
    cout << a;
    return 0;
}
```



C	o	m	p	u	t	e	r
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]

用 cout 输出字符数组

```
#include <iostream>
using namespace std;
int main()
{
        char weekday[7][11] = { "Sunday", "Monday",
"Tuesday", "Wednesday", "Thursday", "Friday",
"Saturday" };
        for (int i = 0; i < 7; i++)
                cout << weekday[i] << endl;
        return 0;
}
```

S	u	n	d	a	y	\0	\0	\0	\0	\0
M	o	n	d	a	y	\0	\0	\0	\0	\0
T	u	e	s	d	a	y	\0	\0	\0	\0
W	e	d	n	e	s	d	a	y	\0	\0
T	h	u	r	s	d	a	y	\0	\0	\0
F	r	i	d	a	y	\0	\0	\0	\0	\0
S	a	t	u	r	d	a	y	\0	\0	\0

```
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
```



（3）一串字符的输入

回顾：单个字符串的输入

```
#include <iostream>
using namespace std;
int main()
{
        char c;
        cout << "enter a sentence:" << endl;
        while (cin >> c)
                cout << c;
        return 0;
}
```

```
enter a sentence:
abc def g
abcdefg
```

方法一：直接用cin输入字符串

```
#include <iostream>
using namespace std;
int main()
{
        char str[10];
        cout << "enter a sentence:" << endl;
        while (cin >> str)
                cout << str <<endl;
        return 0;
}
```

```
enter a sentence:
abc def g
abc
def
g
^Z
```

方法一：直接用cin输入字符串

```
#include <iostream>
using namespace std;
int main()
{
        char str[10];
        cout << "enter a sentence:" << endl;
        while (cin >> str)
                cout << '-' << str <<endl;
        return 0;
}
```

```
enter a sentence:
abc def g
-abc
-def
-g
```

方法一：直接用cin输入字符串

```
#include <iostream>
using namespace std;
int main()
{
        char str[10];
        cout << "enter a sentence:" << endl;
        while (cin >> str)
                cout << str << endl;
        return 0;
}
```

```
enter a sentence:
abcdefghijklmnopqrstuvwxyz
abcdefghijklmnopqrstuvwxyz
^Z
```

方法一：直接用cin输入字符串

```
#include <iostream>
using namespace st
int main()
{
        char str1[10], str2[10];
        cout << "enter a sentence:" << endl;
        while (cin >> str1 >> str2)
                cout << str1 << str2 << endl;
        return 0;
}
```

```
enter a sentence:
hhahahahahahhahahahahahhahahha
17823817236817263871263817 23
hhahahahah1782381723681726387126381723
```

方法一：直接用cin输入字符串

```
#include <iostream>
using namespace std;
int main()
{
        char str[10];
        cout << "enter a sentence:" << endl;
        while (cin >> str)
                cout << '-' << str << endl;
        return 0;
}
```

```
enter a sentence:
sad sadsad sad joijdoasdalsd oajsdoasjndjklasdlkammxoq
-sad
-sadsad
-sad
-joijdoasdalsd
-oajsdoasjndjklasdlkammxoq
```

方法二：用cin.get( )函数输入

- 有3个参数的get函数

```
cin.get(ch, 10,'\n');
```

  - 读取10-1个字符（包含空格），赋给指定的字符数组;
  - 如果在读取10-1个字符之前，遇到指定的终止字符'\n'，则提前结束读取；（如果第3个参数没有指定，则默认为'\n'
  - 读取成功返回非0值(真)，如失败(遇文件结束符)则返回0值(假)。

方法二：用cin.get( )函数输入

```
#include <iostream>
using namespace std;
int main()
{
        char ch[20];
        cout << "enter a sentence:" << endl;
        cin.get(ch, 10);
        cout << ch << endl;
        return 0;
}
```

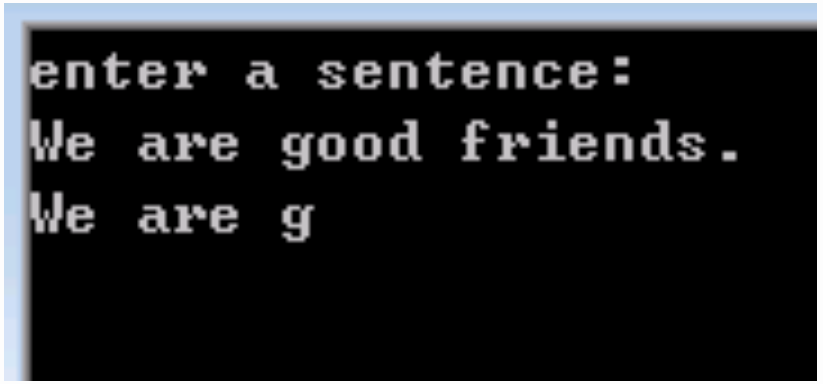
```
enter a sentence:
Sunday
Sunday
请按任意键继续. . .
```

```
enter a sentence:
abcdefghijklmnopqrst
abcdefghi
```

```
enter a sentence:
hello hello hello
hello hel
```

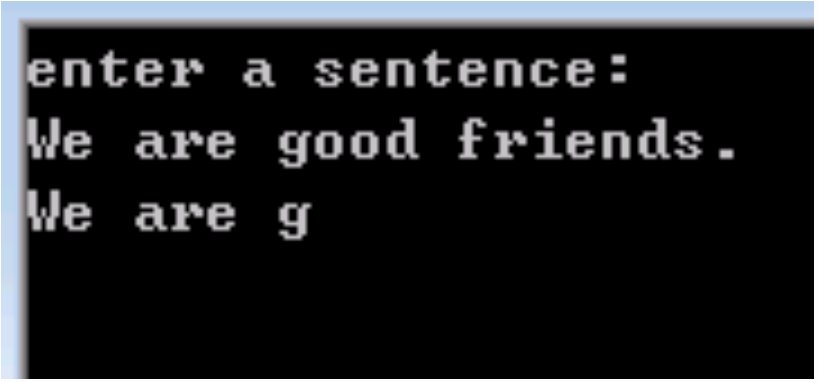
方法二：用cin.get( )函数输入

```
#include <iostream>
using namespace std;
int main()
{
        char ch[20];
        cout << "enter a sentence:" << endl;
        cin.get(ch, 10, 'o'); //指定终止符为'o'
        cout << ch << endl;
        return 0;
}
```



方法三：用cin.getline( )函数输入

```
#include <iostream>
using namespace std;
int main()
{
        char ch[20];
        cout << "enter a sentence:" << endl;
        cin.getline(ch, 10, 'o'); //指定终止符为'o'
        cout << ch << endl;
        return 0;
}
```



方法三：用cin.getline( )函数输入

- getline与get的区别
  - getline遇到终止标志字符时结束，缓冲区指针移到终止标志字符之后；
  - get遇到终止字符是停止读取，指针不移动



We are good friends.

cin.get( )

We are good friends.

cin.getline( )

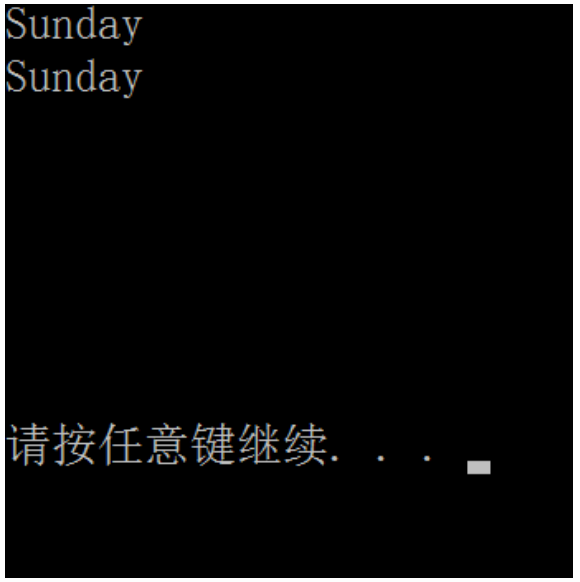
方法三：用cin.getline( )函数输入

```
#include <iostream>
using namespace std;
int main()
{
        char weekday[7][11];
        for (int i = 0; i < 7; i++)
                cin.getline(weekday[i], 11);
        for (int i = 0; i < 7; i++)
                cout << weekday[i] << endl;
        return 0;
}
```

```
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
```

方法三：用cin.getline( )函数输入

```
#include <iostream>
using namespace std;
int main()
{
        char weekday[7][11];
        for (int i = 0; i < 7; i++)
                cin.get(weekday[i], 11);
        for (int i = 0; i < 7; i++)
                cout << weekday[i] << endl;
        return 0;
}
```



方法三：用cin.getline( )函数输入

```
#include <iostream>
using namespace std;
int main()
{
        char weekday[7][11];
        for (int i = 0; i < 7; i++)
        {       cout << i << endl;
                cin.get(weekday[i], 11);}
        for (int i = 0; i < 7; i++)
                cout << weekday[i] << endl;
        return 0;
}
```

```
0
Sunny
1
2
3
4
5
6
Sunny
```

一个需要特别关注的程序

```
#include<iostream>
using namespace std;
int main()
{
        char a[10][10];
        int n = 0;
        cin >> n;
        for (int i = 0; i < n; i++)
                cin.getline(a[i], 10);
        for (int i = 0; i < n; i++)
                cout << a[i] << endl;
        return 0;
}
```

```
7
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday

Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
```

Pass!

```
#include<iostream>
using namespace std;
int main()
{
        char a[10][10];
        int n = 0;
        cin >> n;
        cin.ignore( ); //添加一条语句
        for (int i = 0; i < n; i++)
                cin.getline(a[i], 10);
        for (int i = 0; i < n; i++)
                cout << a[i] << endl;
        return 0;
}
```





字符串的应用

例1 字符串加密

- **字符串加密**
  - 输入一个字符串，把每个字符变成它后续字符，如果是‘Z’或者‘z’，则变成’A’或’a’.空格则不变。然后将变换后的字符串输出；
  - 要求能够接受连续输入；

abcdefghijklmnopqrstuvwxyz

```
hello
ifmmp
nice to meet u.
ojdf up nffu v/
do you like c++
ep zpv mjlf d,,
no?
op@
bye
czf
^Z
Press any key to continue
```



例1 字符串加密

- 思路：
  - 读入字符串（想一下以什么方式输入？）
  - 从字符头到尾循环：
    - 是'Z'则直接赋值'A'，跳过以下步骤
    - 是'z'则直接赋值'a'，跳过以下步骤
    - 空格不做处理，跳过以下步骤
    - 其他字符++ // 就是ASCII表上加一
  - 输出新字符串；

例1 字符串加密

```
#include <iostream>
using namespace std;
int main()
{
        char str[200];
        while (cin.getline(str, 200))
        {
                for (int i = 0; str[i] != '\0'; i++)
                {
                        if (str[i] == 'Z')
                        {
                                str[i] = 'A';  continue;
                        }
                        if (str[i] == 'z')
                        {
                                str[i] = 'a';  continue;
                        }
                        if (str[i] == ' ')
                                continue;
                        str[i]++;
                }
                cout << str << endl;
        }
        return 0;
}
```

```
hello
ifmmp
nice to meet u.
ojdf up nffu v/
do you like c++
ep zpv mjlf d,,
no?
op@
bye
czf
^Z
```

例2 字符串连接

- 问题：
  - 输入两个字符串，将其中较短的串接到较长的串的后面。
- 要求：
  - 不使用系统函数 strcat

    (在 #include <cstring>中)
  - 每个输入的串的长度不超过20。

例2 字符串连接

定义：

```
char str1[40],  str2[40];
cin.getline(str1,20);  cin.getline(str2,20);
```

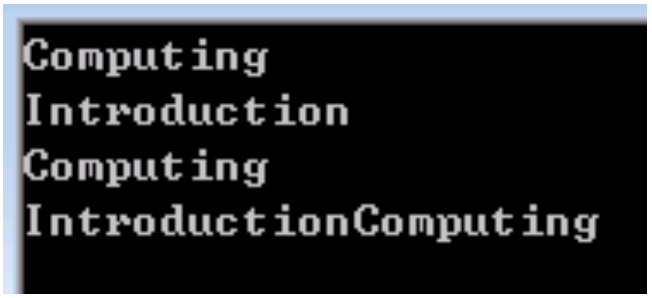
计算长度：

```
for (len1 = 0; str1[len1] != '\0'; len1++);
for (len2 = 0; str2[len2] != '\0'; len2++);
```

拼接：第一个串的下标指向最后一个元素之后
第二个串的下标指向第一个元素。

```
for(len2=0;str1[len2]!='\0';len2++);
        str1[len1++]=str2[len2];
        str1[len1]='\0';      //不加上\0,就不是一个字符串。
```

```
#include <iostream>
using namespace std;
int main()
{
        int len1, len2; char str1[40], str2[40];
        cin.getline(str1, 20); cin.getline(str2, 20);
        for (len1 = 0; str1[len1] != '\0'; len1++);
        for (len2 = 0; str2[len2] != '\0'; len2++);
        if (len1 >= len2)
        {
                for (len2 = 0; str2[len2] != '\0'; len2++)
                        str1[len1++] = str2[len2];
                str1[len1] = '\0';
        }
        else
        {
                for (len1 = 0; str2[len1] != '\0'; len1++)
                        str2[len2++] = str1[len1];
                str2[len2] = '\0';
        }
        cout << str1 << endl;       cout << str2 << endl;
        return 0;
}
```



插入：字符数组常用操作

```
#include <iostream>
#include <string>
using namespace std;
int main()
{   char str1[20],str2[20];
    cin.getline(str1,20);
    strcpy(str2,str1);
    cout << str1 <<endl; //数组名就是字符串名
    cout << str2 <<endl;
}
```



例3 统计单词数

- 问题：
  - 输入一个英文句子（不超过80个字母），统计其中有多少个单词，单词之间用空格分开。



例3 统计单词数

Everything Should Be Made as Simple as Possible, But Not Simpler.

- Albert Einstein

例3 统计单词数

```
#include <iostream>
using namespace std;
int main()
{
        char str[80];
        int num = 0, flag = 0;
        cin.getline(str, 80);
        for (int i = 0; str[i] != '\0'; i++)
        {
                if (str[i] == ' ')
                        flag = 0;
                else if (flag == 0)
                {
                        flag = 1; num++;
                }
        }
        cout << "字符串中有" << num << "个单词" << endl;
        return 0;
}
```

```
He is one of my good friends.
字符串中有7个单词
```



思考题

- 从标准输入输出流的原理出发
- 如何处理文件输入输出
  - https://stackoverflow.com/
  - https://www.cplusplus.com/



好好想想,有没有问题?

谢　谢　！